

Proyecto Integrador: Primera Entrega

Derivative-free optimal multiple-root finding family of iterative methods

Métodos Numéricos

Integrantes:

Ailyn Sofía Gómez Rodríguez

Julián Camilo Jiménez

Julián Steven Duarte

Docente:

Edwin Fernando Muñoz Chipatecua

1. Comprensión del Artículo Científico

1.1. El problema fundamental: ¿por qué es difícil hallar raíces múltiples?

En computación científica y Ciencia de Datos, un problema elemental consiste en resolver ecuaciones no lineales $g(x) = 0$. Geométricamente, una raíz simple α corta el eje horizontal con una pendiente clara ($g'(\alpha) \neq 0$), permitiendo que los métodos iterativos identifiquen con facilidad la dirección hacia la solución.

Sin embargo, en diversos problemas físicos y termodinámicos aparecen soluciones con **multiplicidad** $m \geq 2$, donde tanto la función como sus primeras $m - 1$ derivadas se anulan en la raíz:

$$g(\alpha) = 0, \quad g'(\alpha) = 0, \quad \dots, \quad g^{(m-1)}(\alpha) = 0, \quad \text{pero } g^{(m)}(\alpha) \neq 0. \quad (1)$$

Gráficamente, esto aplana por completo la curva alrededor de la raíz. Al no existir una pendiente definida en un intervalo amplio, los algoritmos numéricos sufren para distinguir la ubicación exacta del cero.

1.2. ¿Por qué fallan los métodos clásicos y qué dilema surge en la práctica?

El método iterativo por excelencia es Newton-Raphson ($x_{k+1} = x_k - \frac{g(x_k)}{g'(x_k)}$), el cual posee convergencia cuadrática ($p = 2$) en raíces simples. No obstante, frente a una raíz con multiplicidad $m \geq 2$, tanto $g(x_k)$ como $g'(x_k)$ se aproximan a cero a la vez. Al dividir dos cantidades infinitesimales, el avance se frena drásticamente: la rapidez cuadrática se degrada a convergencia lineal ($p = 1$), exigiendo cientos de iteraciones para ganar apenas unos decimales.

Para solucionarlo, Schröder propuso el **Newton modificado**, multiplicando el paso por la multiplicidad m :

$$x_{k+1} = x_k - m \frac{g(x_k)}{g'(x_k)}. \quad (2)$$

Aunque esto restaura el orden 2, acarrea un grave inconveniente: **obliga a conocer y evaluar la derivada analítica $g'(x)$ en cada iteración**. En Ciencia de Datos, muchas funciones provienen de simulaciones, algoritmos de caja negra o datos discretos donde no existe una fórmula analítica para derivar. Intentar aproximar $g'(x)$ con diferencias finitas $\frac{g(x+h)-g(x)}{h}$ desencadena el fenómeno de **cancelación catastrófica** estudiado en clase: en aritmética de punto flotante (estándar IEEE-754 con precisión doble), restar dos valores de la función casi idénticos destruye los dígitos significativos y amplifica brutalmente el error numérico.

1.3. La propuesta del artículo: orden cuatro sin derivadas analíticas

Frente a esto, el artículo de Jerezano, Chicharro y Garrido-Sáez (2026) propone una familia de métodos iterativos en dos pasos que **prescinde de derivadas analíticas** y alcanza un **orden de convergencia cuatro** ($p = 4$), cuadruplicando la cantidad de cifras decimales correctas en cada iteración ($e_{k+1} \approx Ce_k^4$).

¿Cómo logra el método esta alta velocidad sin derivadas?

1. **Aproximación secante adaptativa:** Para evitar $g'(x_k)$, el algoritmo calcula un punto auxiliar sumándole al iterado actual una cantidad cuadrática: $w_k = x_k + \beta[g(x_k)]^2$. Al acercarse a la raíz, $g(x_k)$ se desvanece suavemente. Así, la diferencia dividida secante $g[w_k, x_k] = \frac{g(w_k)-g(x_k)}{w_k-x_k}$ aproxima fielmente a la primera derivada sin necesidad de fijar un paso artificial h ni sufrir cancelación severa. Con esto se obtiene un predictor intermedio y_k .
2. **Aceleración con función de peso:** Con y_k , el método evalúa una variable de control adimensional $t_k = (g(y_k)/g(x_k))^{1/m}$, la cual tiende rápidamente a cero. Mediante series de Taylor del error, los autores demostraron que si la función de peso $H(t)$ cumple en el origen:

$$H(0) = 1, \quad H'(0) = 1, \quad H''(0) = 4, \quad |H'''(0)| < \infty, \quad (3)$$

se cancelan exactamente los errores de primer, segundo y tercer orden, elevando la convergencia a orden 4.

Optimalidad de Kung-Traub: La conjetura de Kung y Traub (1974) establece que el orden máximo para un método sin memoria con d evaluaciones funcionales es $p_{\max} = 2^{d-1}$. Como este método evalúa la función solo 3 veces ($g(x_k), g(w_k), g(y_k)$) y ninguna derivada ($d = 3$), su cota máxima teórica es $2^{3-1} = 4$. Al alcanzar exactamente orden 4, la familia se clasifica como **óptima**.

1.4. Supuestos y limitaciones del método

El análisis crítico del artículo identifica tres limitaciones técnicas indispensables:

- **Conocimiento de la multiplicidad m :** El valor de m interviene de forma explícita en las ecuaciones. Si se introduce una multiplicidad incorrecta, el método pierde el orden 4 y diverge.
- **Sensibilidad a la semilla inicial x_0 y al parámetro β :** Si x_0 se elige muy lejos de la raíz o β es muy grande, la perturbación puede provocar oscilaciones. Los autores recomiendan fijar $\beta = 0.01$.
- **Rama principal en potencias fraccionarias complejas:** En $t_k = (g(y_k)/g(x_k))^{1/m}$, cuando m es par o la división es negativa, la raíz m -ésima arroja valores complejos. Numéricamente debe emplearse la rama principal del logaritmo ($-\pi < \text{Arg}(z) \leq \pi$) para evitar discontinuidades.

2. Formulación del Problema Computacional

2.1. Contexto en Ciencia de Datos y objetivo del proyecto

En Ciencia de Datos, optimizar funciones de pérdida complejas equivale a buscar puntos estacionarios donde el gradiente se anula ($\nabla L(\theta) = 0$). Cuando las funciones provienen de simuladores de caja negra sin gradientes analíticos, y donde mínimos planos simulan raíces múltiples, los métodos tradicionales sin derivadas convergen lentamente o sufren cancelación numérica.

El **objetivo del proyecto** consiste en implementar en Python, de forma autónoma, clara y reproducible (sin solvers de caja negra de SciPy), las familias óptimas de orden 4 libres de derivadas de Jerezano et al. (2026), verificar experimentalmente su tasa de convergencia en raíces múltiples y comparar su desempeño frente a los métodos clásicos vistos en clase.

2.2. Formulación matemática del algoritmo iterativo

El esquema predictor-corrector en dos pasos se expresa formalmente como:

$$\text{Paso 1 (Perturbación adaptativa): } w_k = x_k + \beta[g(x_k)]^2, \quad (4)$$

$$\text{Paso 2 (Predictor intermedio): } y_k = x_k - m \frac{g(x_k)}{g[w_k, x_k]}, \quad (5)$$

$$\text{Paso 3 (Corrector final de orden 4): } x_{k+1} = x_k - m H(t_k) \frac{g(x_k)}{g[w_k, x_k]}. \quad (6)$$

donde la pendiente secante $g[w_k, x_k]$ y la variable de control t_k se calculan como:

$$g[w_k, x_k] = \frac{g(w_k) - g(x_k)}{w_k - x_k}, \quad t_k = \left(\frac{g(y_k)}{g(x_k)} \right)^{1/m} = \exp \left(\frac{1}{m} \text{Log} \left(\frac{g(y_k)}{g(x_k)} \right) \right). \quad (7)$$

Las dos familias a implementar (M_1 y M_2):

1. **Método M_1 (Acelerador Polinomial):** $H_1(t) = 1 + t + 2t^2$. Es una alternativa sumamente económica, basada únicamente en sumas y productos elementales.
2. **Método M_2 (Acelerador Racional):** $H_2(t) = \frac{1-t}{1-2t}$. En el análisis dinámico complejo posee la ventaja de carecer de puntos fijos extraños atractores, evitando converger hacia raíces falsas.

2.3. Variables del modelo y resultados esperados a reproducir

La Tabla 1 detalla el significado computacional de cada símbolo del esquema:

Resultados que esperamos reproducir: Durante el semestre reproduciremos en Python los experimentos del artículo para comprobar: (i) la evolución numérica de los iterados x_k y sus residuos $|g(x_k)|$; (ii) la confirmación experimental del orden de convergencia 4 mediante la fórmula ACOC; y (iii) la comparación cuantitativa de tiempo de CPU y estabilidad frente a los esquemas clásicos del curso.

Cuadro 1: Variables y parámetros del modelo iterativo.

Símbolo	Nombre	Tipo	Significado y función en el algoritmo
x_k	Iterado actual	Real/Complejo	Estimación actual de la raíz en el paso k .
w_k	Punto auxiliar	Real/Complejo	Desplazamiento adaptativo para evaluar la recta secante sin paso fijo.
β	Factor de escala	Constante ($\beta = 0.01$)	Regula la amplitud del paso secante para mantener la estabilidad.
$g[w_k, x_k]$	Diferencia dividida	Real/Complejo	Pendiente secante que sustituye numéricamente a la derivada analítica.
m	Multiplicidad	Entero ($m \geq 2$)	Multiplicidad de la raíz que neutraliza el aplanamiento de la curva.
y_k	Predictor intermedio	Real/Complejo	Aproximación preliminar que reduce el residuo para alimentar a t_k .
t_k	Cociente de control	Complejo	Variable adimensional que mide el avance hacia la solución ($t_k \rightarrow 0$).
$H(t)$	Función de peso	Función analítica	Mecanismo acelerador que cancela los errores inferiores y otorga orden 4.
x_{k+1}	Iterado nuevo	Real/Complejo	Nueva aproximación refinada obtenida al concluir la iteración.
α	Raíz analítica	Real/Complejo	Cero exacto de la ecuación ($g(\alpha) = 0$).

3. Obtención de la Información y Problemas de Prueba

3.1. Naturaleza de la información requerida

En estricto cumplimiento de la guía, definimos con total precisión la procedencia de la información: **no se trabajará con datos empíricos ni series con ruido estocástico**. El proyecto utilizará **funciones matemáticas analíticas deterministas** de prueba (*benchmark problems*) provenientes de problemas físicos y de ingeniería documentados en el artículo base.

Esta decisión metodológica se justifica plenamente: al disponer de funciones cerradas con raíces exactas conocidas (α), es posible calcular con absoluta certeza el error numérico verdadero $|x_k - \alpha|$, evaluar el residuo exacto $|g(x_k)|$ y certificar si los métodos alcanzan la tasa teórica de orden 4 sin interferencias causadas por ruido de medición.

3.2. Descripción de las funciones benchmark seleccionadas

Para validar las implementaciones, utilizaremos los tres problemas de prueba presentados en el artículo de Jerezano et al. (2026), los cuales abarcan distintas multiplicidades y dificultades computacionales:

1. Problema 1 (Ecuación de Van der Waals en Termodinámica):

$$g_1(x) = x^3 - 5.22x^2 + 9.0825x - 5.2675 = (x - 1.75)^2(x - 1.72). \quad (8)$$

Modela gases reales. Presenta una raíz doble ($m = 2$) en $\alpha = 1.75$ y una raíz simple vecina en 1.72. La dificultad crítica es su cercanía extrema ($\Delta = 0.03$): si el método es inestable o la semilla es lejana, puede ser capturado por la raíz simple vecina.

2. Problema 2 (Ley de Radiación Térmica de Planck):

$$g_2(x) = \left(e^{-x} - 1 + \frac{x}{5}\right)^3. \quad (9)$$

Calcula la longitud de onda de máxima densidad espectral en la radiación de cuerpo negro. Posee una raíz triple ($m = 3$) en $\alpha \approx 4.965114$. Al ser trascendente con términos exponenciales, evalúa la estabilidad fuera del ámbito algebraico simple.

3. Problema 3 (Polinomio Característico en Álgebra Lineal):

$$g_3(x) = (x - 3)^4(x - 8)(x - 5)(x - 4)(x - 1)(x + 1). \quad (10)$$

Ecuación característica de una matriz 9×9 . Posee una raíz cuádruple ($m = 4$) en $\alpha = 3$ y coeficientes muy grandes, exigiendo un estricto control de errores de redondeo y riesgos de overflow o underflow.

Cuadro 2: Características y parámetros de las funciones de prueba del proyecto.

Función de prueba $g(x)$	Contexto	Raíz α	Mult. (m)	Semillas (x_0)	Desafío principal	numérico
$g_1(x) = (x - 1.75)^2(x - 1.72)$	Termodinámica	1.75	2	1.9, 3.0	Extrema cercanía a raíz simple ($\Delta = 0.03$).	
$g_2(x) = (e^{-x} - 1 + x/5)^3$	Física Térmica	≈ 4.96511	3	5.3, 10.0	Presencia de términos exponenciales trascendentes.	
$g_3(x) = (x - 3)^4 \prod (x - \lambda_i)$	Álgebra Lineal	3.0	4	2.8, 3.5	Multiplicidad 4 y coeficientes de gran escala.	

4. Plan de Implementación y Criterios de Comparación

4.1. Métodos numéricos a implementar y justificación de la comparativa

Para evaluar los aportes del artículo frente a los conceptos vistos en clase, implementaremos en Python dos grupos de algoritmos:

- Métodos del artículo base (Familias de orden 4 sin derivadas):** Métodos M_1 (acelerador polinomial) y M_2 (acelerador racional), ambos con parámetro $\beta = 0.01$.
- Métodos de contraste vistos en el curso:**
 - Newton modificado de Schröder* ($p = 2$): Método clásico con derivada analítica $g'(x)$, útil para evaluar qué se gana o se pierde en precisión y tiempo frente a la diferencia secante adaptativa.
 - Steffensen modificado* ($p = 2$): Método clásico sin derivadas con diferencia dividida fija, permitiendo comprobar si el diseño adaptativo con $\beta[g(x)]^2$ y $H(t)$ aporta ventajas frente a esquemas sin derivadas estándar.

4.2. Criterios técnicos para la comparación de resultados

Para contrastar el desempeño de cada método en las funciones benchmark, utilizaremos cinco criterios cuantitativos:

- Error numérico y residuo final ($|g(x_{k+1})|$ y distancia entre iterados $|x_{k+1} - x_k|$):** Evaluaremos el residuo final y el decremento entre iterados empleando precisión doble estándar (`float64`, $\varepsilon_M \approx 2.22 \times 10^{-16}$) y aritmética de precisión múltiple (`mpmath` con 200 dígitos decimales), confirmando si los métodos resuelven la raíz con extrema exactitud sin sufrir inestabilidad numérica.
- Convergencia y orden computacional empírico (ACOC):** Calcularemos el Orden de Convergencia Computacional Aproximado formulado por Cordero y Torregrosa: $ACOC \approx \frac{\ln |(x_{k+1} - x_k)/(x_k - x_{k-1})|}{\ln |(x_k - x_{k-1})/(x_{k-1} - x_{k-2})|}$. Un valor empírico cercano a 4.0000 confirmará en la práctica que el error se reduce a la cuarta potencia en cada iteración.
- Estabilidad numérica y sensibilidad a la semilla inicial (x_0):** Evaluaremos la robustez del método probando semillas cercanas y lejanas (como $x_0 = 1.9$ y $x_0 = 3.0$ en el Problema 1), registrando si converge limpiamente o si se presentan divergencias u oscilaciones.
- Eficiencia computacional (Índice de Kung-Traub $EI = p^{1/d}$):** Balancea el orden de convergencia (p) frente al número de evaluaciones de función (d) por iteración. Mientras Newton modificado ($p = 2, d = 2$) tiene $EI = 2^{1/2} \approx 1.414$, las familias propuestas ($p = 4, d = 3$) alcanzan $EI = 4^{1/3} \approx 1.587$, confirmando teóricamente un mayor rendimiento por operación ejecutada.
- Tiempo de ejecución en procesador (CPU):** Mediremos en Python (utilizando el módulo `time` y promediando 50 ejecuciones independientes) el tiempo real en milisegundos que tarda la computadora en ejecutar cada algoritmo hasta converger, comprobando si la ausencia de derivadas analíticas se traduce en un menor tiempo de cómputo en hardware.

Declaración sobre el uso de herramientas de Inteligencia Artificial

En cumplimiento de las normas de integridad académica de la Universidad Externado de Colombia y las directrices del curso Métodos Numéricos, los integrantes del equipo declaramos que utilizamos herramientas de Inteligencia Artificial generativa exclusivamente como apoyo técnico para la diagramación del documento en $\text{\LaTeX}$ y la revisión de estilo gramatical. Los autores comprendemos integralmente, sustentamos y asumimos la total autoría y responsabilidad de los fundamentos matemáticos, la formulación computacional y la metodología desarrollada en este informe.

Referencias

- [1] Jerezano, J. H., Chicharro, F. I., & Garrido-Sáez, N. (2026). Derivative-free optimal multiple-root finding family of iterative methods. *Computational and Applied Mathematics*, 45:304.
- [2] Kung, H. T., & Traub, J. F. (1974). Optimal order of one-point and multipoint iteration. *Journal of the ACM*, 21(4), 643–651.
- [3] Cordero, A., & Torregrosa, J. R. (2007). Variants of Newton’s method using fifth-order quadrature formulas. *Applied Mathematics and Computation*, 190(1), 686–698.
- [4] Turner, P. R. (2010). *Guide to Scientific Computing*. Springer-Verlag.